

unicode 术语策略：与 Pi、OpenCode 趋同还是引用？

论述 unicode 作为自主开发的 Coding Agent Harness，在术语定义上应如何对待 Pi、OpenCode 两家已在行业内形成代表性与影响力的第三方项目。

本文档为策略与设计原则，不替代具体词条表；实现层对照见 GLOSSARIES_COMPARISON.md。

项	说明
文档类型	术语与对外叙事策略
路径	docs/technologies/TERMINOLOGY_ALIGNMENT_STRATEGY.md
最后更新	2026-05
相关 Issue	若据此调整公开 API 命名或大规模文档改写，建议单独立项跟踪

1. 问题陈述

unicode 在架构上以 Pi 为哲学与机制参照（见 UNCODE_PI_ALIGNMENT_AND_EVALUATION.md），同时与 OpenCode 存在大量概念重叠（Agent 循环、会话、工具、流式事件、Server 交付等，见 OPENCODE_VS_PI.md）。

由此产生策略分歧：

取向	主张	表面收益
趋同	类型名、事件名、文档用语尽量与 Pi 或 OpenCode 一致	降低学习成本、「听起来像主流」
引用	保留 unicode 自有命名，在文档中做对照表与链接	保留 Rust/工程诚实、避免虚假等价
混合	概念层趋同、API 层自有、产品层选择性借鉴	平衡生态与实现自由

核心问题：趋同到什么程度才值得？引用到什么程度才够用？

2. 三家项目的术语角色（应先分清）

在讨论「要不要趋同」之前，应明确三者 in unicode 话语体系中的不同权重：

项目	与 unicode 的关系	术语表角色	建议权重
Pi	显式对齐的架构参照系（Harness 分层、双环、Steering 三通道、逻辑会话树）	PI_TECHNOLOGIES_GLOSSARY	高——概念与机制对齐的主轴

项目	与 uncode 的关系	术语表角色	建议权重
OpenCode	竞品 / 平行产品（同为 TS Agent 产品，哲学与 Pi 有分野）	OPENCODE_TECHNOLOGIES_GLOSSARY	中 — 产品能力与交付对照，非 uncode 第一命名来源
Harness 行业综述	跨项目概念语言（Agent = Model + Harness、Compaction、P+G+E）	HARNESSENGINEERINGGLOSSARY	高 — 对外宣讲、培训、与非 Pi 读者沟通

结论（前提）：不存在「同时与 Pi 和 OpenCode 在 API 层全面趋同」的可行方案——二者在 MCP、子 Agent、会话存储、事件命名上**已分叉**。uncode 若「趋同」，必须先回答：**趋同 Pi 还是趋同 OpenCode？**本仓库既有文档已选择 **Pi 为机制主轴**，OpenCode 为**对照样本**，这一前提应写入术语策略。

3. 三种策略的利弊

3.1 策略 A：实现层全面趋同（不推荐为默认）

做法：Rust 公开 API 尽量采用 Pi 英文名（如 `convert_to_llm`、`AgentMessage`）或 OpenCode 名（`SessionProcessor`、`session.next.*`）。

优点	缺点
读过 Pi 文档的开发者上手快	语言与栈不适配： Rust 惯用 <code>snake_case</code> 、枚举扩展方式与 TS declaration merging 不同
对外可称「Rust 版 Pi」	与 OpenCode 无法同时趋同 ，易造成叙事混乱
	虚假等价： 同名不同义（见 §4）时损害信任
	上游更名即被动： Pi/OpenCode 演进会拖拽 uncode 破坏性变更
	OpenCode 事件名 <code>session.next.*</code> 与 uncode <code>AgentEvent</code> 模型不一致，硬抄无工程收益

适用例外：逻辑模型已与 Pi 同构的类型（如 `SessionEntry` 树、`AgentHarness`、`steering/follow_up/next_turn`）——这些在 uncode 中**已趋同概念**，无需再改名为 Pi 的 TS 专名。

3.2 策略 B：仅引用、完全自有命名（偏保守）

做法：代码与对外文档只用 uncode 术语；Pi/OpenCode 仅出现在「其他项目」脚注。

优点	缺点
品牌与实现边界清晰	重复发明轮子 ：行业已共识的 Harness/Turn/Compaction 若另造中文名，增加协作成本
无「山寨」观感	贡献者与 FDE 缺少共同坐标 ，培训成本高
	与仓库内「对齐 Pi」的技术文档 自相矛盾

适用：实现**专有**、且与 Pi/OpenCode 无稳定一一对应的符号（如 `SurrealSessionStore`、`#[tool]`、`broadcast::Sender<AgentEvent>`）。

3.3 策略 C：分层混合（推荐）

做法：按抽象层决定「趋同 / 引用 / 自有」：

L0 行业概念 (Harness、Agent、Tool、Compaction、Sandbox)
→ 与 Pi/OpenCode 共用英文概念词；中文文档与 HARNESS 综述表对齐
L1 机制概念 (Turn、Steering、Follow-up、Session 树、AgentHarness)
→ 与 Pi 对齐命名与语义；文档写「同 Pi 的 X」
L2 实现 API (Rust 类型、函数、c rate)
→ uncode 自有 idiomatic 命名；在术语表注明 Pi/OpenCode 映射
L3 产品交付 (TUI、Platform、存储路径)
→ 自有；OpenCode 仅作 UX/能力_benchmark（如 scrollbar 格式）

优点	缺点
兼顾生态可读性与 Rust 工程诚实	需维护对照表（已有四表 + 本文）
与现有 <code>UNCODE_PI_ALIGNMENT</code> 叙事一致	新贡献者需读一篇策略文（本文）
OpenCode 可作能力对照而不绑架命名	

这是 uncode 当前文档体系已在实践的方向；本文将其**显式化**为策略，避免未来贡献时随意改名或随意造词。

4. 必须警惕的「同名异义」

趋同的最大风险不是法律问题，而是**读者以为 API 可移植，实际不能**。

术语	Pi / uncode 含义	OpenCode / Harness 含义	策略
Steering	运行时中途 纠偏消息队 列	(OpenCode 无三队列专名)	uncode 保留 steering；文档勿与 Harness「人改 Harness 闭环」混用

术语	Pi / uncode 含义	OpenCode / Harness 含义	策略
Steering	—	Harness 综述： 人改 Harness	对外培训用「运行时 Steering」vs「治理 Steering」分称
Agent	Agent 类 / 配置	产品 Agent 角色 (build/plan)	uncode 文档区分 Agent 类 vs Agent 产品角色
Session	树状逻辑会话	SQLite session 行 + message/part	uncode 强调 逻辑 SessionEntry vs Surreal 物理存储
Compaction	概念一致	事件 compaction.* vs SessionEntry::Compaction	概念趋同即可，事件名不必抄 session.next.*
MCP	Pi：非主路径	OpenCode：一等公民	uncode 不为 趋同 OpenCode 而改文档立场；若引入须单独立项

原则：概念可趋同，**禁止**在未实现等价行为时复用对方**专有**名（如 **SessionProcessor**、**JsonlSessionStorage**）。

5. 推荐策略（结论）

5.1 总原则

- 概念与机制：**向 Pi 趋同，向行业（Harness 综述）接轨，向 OpenCode 对照但不默认趋同 API。
- **实现符号：**Rust idiomatic 自有命名；在 **UNCODE_TECHNOLOGIES_GLOSSARY** 维护「uncode ↔ Pi ↔ OpenCode」映射列（可逐步补全）。
- **对外叙事：**先说行业语言（Harness、Agent Coding），再说「逻辑对齐 Pi」；OpenCode 用于能力对比（Server、工具广度、MCP），不用于定义 uncode 核心名词。
- 直接引用优于假装自研：**文档与注释中**明确**写出「同 Pi 的 **convertToLlm**」「对标 OpenCode 的 scrollbar」——引用是**诚实**，趋同是**选择**。

5.2 何时「趋同」合适

场景	建议
新文档章节描述 双环、Turn、压缩、分支	用 Pi 同款英文概念词 + 链到 Pi 文档
新 AgentEvent 变体对应 Pi 已有阶段	命名与语义对齐 Pi 事件序列（见 PI_EVENT_SYSTEM ）
对外白皮书、培训	优先 HARNESS_ENGINEERING_GLOSSARY + Pi 机制表
用户可见 CLI/TUI 文案	行业通用中文（会话、工具、压缩），避免 Pi 内部专名

5.3 何时「仅引用、不趋同」合适

场景	建议
Rust 函数/类型公开 API	build_context 而非 convert_to_llm ；在 doc comment 写 /// Pi: convertToLlm

场景	建议
存储与路径	<code>SurrealSessionStore</code> 、 <code>~/.uncode/</code> ，不引用 <code>JsonlSessionStorage / opencode.db</code>
事件传输	保持 <code>AgentEvent</code> 枚举，不改为 <code>session.next.*</code> 字符串枚举
OpenCode 产品功能	MCP 主路径、build/plan、Task 子会话—— 引用对比 ，除非 uncode 产品决策采纳

5.4 何时「直接引用」合适（链接 + 脚注）

- 实现层设计文档首次出现概念时：参见 `Pi : ...`、参见 `OpenCode : ...`。
- `GLOSSARIES_COMPARISON.md` 与四份术语表互链。
- PR / Issue 讨论机制变更时：要求更新对照表一行，避免口头「和 Pi 一样」。

5.5 对 OpenCode 的单独立场

OpenCode 影响力大，但 uncode 已选 **Pi 为架构主轴**，故：

做法	建议
把 <code>SessionProcessor</code> 搬进 uncode 作为类型名	否
在 <code>OPENCODE_VS_PI</code> / Platform 设计中引用其 Server、工具集、SQLite 产品形态	是
术语表收录 OpenCode 专名供读者查	是（已实现）
为「像 OpenCode」改 uncode 哲学（如默认 MCP）	否，除非经 <code>docs/</code> + Issue 显式决策

6. 文档与代码落地规则

6.1 文档

文档类型	规则
<code>docs/uncode-technologies/*</code>	以 uncode 专名为准；首次出现 Pi 概念时括号注明 Pi 名
<code>docs/pi-technologies/*</code>	描述 Pi，不强行改成 uncode 名
<code>docs/opencode-technologies/*</code>	描述 OpenCode，作为第三方参照
<code>docs/technologies/*</code> 策略/对比	允许横评与命名策略（本文、对齐评价、 <code>OPENCODE_VS_PI</code> ）
中文对外材料	概念层用行业中文；实现层少堆砌英文 API

6.2 代码

层级	规则
----	----

层级	规则
uncode-core 公开类型	稳定、idiomatic；与 Pi 对齐写进 doc comment，不强制同名
内部模块	可更自由；重构不必照顾 Pi 拼写
用户配置键	优先稳定与可读（~/uncode/config.toml），不复制 opencode.json 键名除非兼容需求

6.3 术语表维护

- **新增 uncode 专名** → 写入 UNCODE_TECHNOLOGIES_GLOSSARY，并评估是否在 §「Pi 对应」「OpenCode 对应」加列。
- **Pi/OpenCode 上游改名** → 只更新对应实现表与 GLOSSARIES_COMPARISON，**不强制**改 uncode API。
- **行业新词**（如新的 Harness 范式）→ 先更新 HARNESS_ENGINEERING_GLOSSARY，再决定 uncode 是否采纳概念。

7. 决策矩阵（速查）

面对「这个词要不要改成和 Pi/OpenCode 一样？」：

是否 Pi 机制文档中的核心概念（Turn、Compaction、Steering、Harness）？

- └ 是 → 文档与注释用同一英文概念；Rust API 可自有名 + doc 映射
- └ 否 → 继续

是否仅为 OpenCode 产品/实现专名（SessionProcessor、session.next.*）？

- └ 是 → 不趋同；在对比文档与 OpenCode 术语表引用
- └ 否 → 继续

是否 Rust/存储/事件模型独有？

- └ 是 → 完全自有命名
- └ 否 → 考虑行业综述表是否已有通用词 → 优先用行业词

8. 与现有资产的关系

本策略**不推翻**当前做法，而是为已有文档体系提供依据：

已有资产	策略下的定位
UNCODE_PI_ALIGNMENT_AND_EVALUATION.md	机制对齐的权威评价（Pi 主轴）
OPENCODE_VS_PI.md	第三方横评； 不 作为 uncode 命名法源
四份术语表 + GLOSSARIES_COMPARISON.md	引用层的操作手册
AgentHarness / SessionEntry / Steering 三通道	已采纳 的概念层趋同示例
SurrealDB / AgentEvent	已采纳 的实现层自有示例

9. 建议的后续动作（可选）

可执行方案已写入 [TERMINOLOGY_LAYERED_REFACTOR_PLAN.md](#)（Phase 1–4、crate 清单、PR 检查项、验收标准）。摘要：

- 1. Phase 1：术语表 Pi/OpenCode 列 + [UNCODE_PI_MECHANISM_MAP](#) + 系列文档 L1 声明。
- 2. Phase 2：事件 / 会话 / 循环对照矩阵。
- 3. Phase 3：核心 [pub](#) API 的 `/// Pi:` rustdoc（不改符号名）。
- 4. Phase 4（默认不做）：可选 deprecated 别名，须单独立项。
- 5. **不要**发起「全局改名为 Pi 风格」类重构，除非有明确版本边界与迁移说明。

10. 一句话结论

应在概念与机制层与 Pi（及 Harness 行业语言）趋同，在实现 API 层保持 `unicode` 自有命名并通过对照表与文档引用建立桥梁；OpenCode 宜作为能力与产品对照的引用源，不宜作为 `unicode` 核心术语的第二命名法源。

趋同是为了降低理解成本；引用是为了避免虚假等价。二者同时做，但作用在不同抽象层。

相关文档

文档	说明
GLOSSARIES_COMPARISON.md	四份术语表对照
UNCODE_PI_ALIGNMENT_AND_EVALUATION.md	<code>unicode</code> 与 Pi 对齐评价
OPENCODE_VS_PI.md	OpenCode 与 Pi 对比
UNCODE_TECHNOLOGIES_GLOSSARY.md	<code>unicode</code> 术语表
AGENTS.md	协作与文档约定
TERMINOLOGY_LAYERED_REFACTOR_PLAN.md	策略 C 分阶段重构方案

路径：[docs/technologies/TERMINOLOGY_ALIGNMENT_STRATEGY.md](#)